Стандартотворець

Платформа управління lifecycle стандартів у робочих групах державного сектору: робочі групи → стандарти → поетапний план → документи → обговорення → засідання → голосування → завдання, і наскрізно — аудит-журнал.

Прод: https://standart.202ok.online/

Документація

Файл	Про що	Коли читати
`docs/ARCHITECTURE.md`	**Як система влаштована зараз**: модель, ролі, життєві цикли, функціонал	щоб зрозуміти систему
`HANDOFF.md`	Вхід у роботу за 5 хвилин: стек, структура, запуск	перед першим комітом
`CONTINUATION.md`	Поточний стан, що відкрито, блокери	щодня
`docs/DECISIONS.md`	ADR — **чому** саме так	перед тим, як переробляти
`OPS.md`	Деплой, дебаг, відновлення	коли щось впало
`DESIGN.md`	Токени, патерни UI, чекліст екрана	перед новим екраном
`TASKS.md`	Нумерований беклог і історія виконаного	при постановці задач
`CHANGELOG.md`	Що змінювалось по релізах	при розборі регресій
`TZ.md`	Початкове ТЗ (заморожене, травень 2026)	щоб звірити з вимогами
`QA-tests/`	Архів QA-прогонів + регламент	перед QA-циклом

Вимоги

- Node.js 20.x LTS
- pnpm 9.x
- Docker + Docker Compose

Старт для розробки

```
# 1. Клонувати репо та встановити залежності
git clone <repo_url> && cd standardotvoryets
pnpm install

# 2. Налаштувати середовище
cp .env.example .env
# Відредагувати .env (NEXTAUTH_SECRET: openssl rand -base64 32)

# 3. Запустити сервіси (PostgreSQL + Redis + MinIO + MailHog)
docker compose up -d

# 4. Згенерувати Prisma client, синхронізувати схему, засіяти дані
pnpm exec prisma generate   # ПЕРЕД typecheck/lint, інакше tsc сипле implicit-any
pnpm prisma db push         # НЕ migrate dev — міграційних файлів немає (ADR-0004)
pnpm prisma db seed

# 5. Запустити dev-сервер
pnpm dev

# 6. (опційно) Запустити workers для черг BullMQ
pnpm worker
# або всі разом:
pnpm dev:all
```

Відкрити у браузері: http://localhost:3000

Тестовий адмін: `admin@test.ua` / `Admin123!`

Інші сервіси (локально)

Сервіс	URL
Prisma Studio	`pnpm prisma:studio` → http://localhost:5555
MinIO Console	http://localhost:9001 (minioadmin / minioadmin)
MailHog (email)	http://localhost:8025

Корисні команди

```
pnpm typecheck            # перевірка TypeScript
pnpm lint                 # ESLint (--max-warnings 0: один warning блокує деплой)
pnpm format               # Prettier
pnpm test                 # Vitest unit (48 кейсів, 5 файлів)
pnpm test:coverage        # + HTML-звіт у ./coverage/
pnpm test:audit-coverage  # кожна tRPC-мутація має logActivity() (потрібна жива БД)
pnpm test:modules         # integration CRUD через appRouter.createCaller (потрібна БД)
pnpm qa:smoke             # read-only smoke проти staging
pnpm prisma db push       # синхронізувати схему з prisma/schema.prisma
pnpm prisma:studio        # Prisma Studio → localhost:5555
```

> ⚠️ У проєкті **немає** `prisma/migrations/` — схема синхронізується через `db push` (ADR-0004). Тому `pnpm prisma:migrate` і `pnpm prisma:reset` з `package.json` для цього репозиторію не застосовні.

> ⚠️ Тримай репо **поза iCloud** (`~/Documents` / `~/Desktop`) — sync вішає `tsc` / `eslint` / `next build` на `read()` по `node_modules`.

Деплой

Проект — **standalone Docker image**, без зовнішніх залежностей від платформи. Може бути розгорнутий на будь-якому Docker-хості з тих самих ENV-змінних (див. `.env.example`).

Railway (поточний production)

Railway автодетектить `Dockerfile`. Зміни в `main` гілці тригерять автоматичний редеплой. Потрібно тільки заповнити Variables за `.env.example`. Жодних додаткових сервісів — cron, бекапи й сповіщення живуть всередині того ж контейнера.

Fly.io / Render / Coolify / Docker host

```
docker build -t standardotvorets .
docker run -d --name standardotvorets -p 3000:3000 --env-file .env standardotvorets
```

Повний self-hosted стек одною командою

```
cp .env.example .env  # обов'язково встанови NEXTAUTH_SECRET
docker compose --profile fullstack up -d
```

Підіймає app + Postgres + MinIO (S3) + Redis. Перевірка: `http://localhost:3000`.

Cron / scheduler

Усі планові задачі (нагадування про засідання та етапи, тижневий дайджест, нічний бекап БД) виконуються **в тому самому процесі** через `node-cron` — зовнішній планувальник не потрібен. Графіки задані в `src/instrumentation.ts`, часовий пояс `Europe/Kyiv`:

Задача	Розклад
Нагадування / etap-deadlines / vote-closing	Щогодини в `:00` (stage-нагадування лише о 09:00)
Тижневий звіт керівникам	Понеділок 09:00
Дамп БД → S3 (з ретеншеном 30 днів)	Щодня 03:00

ENV-перемикачі: `CRON_DISABLED=1` повністю вимикає планувальник (для сценаріїв з кількома instances), `CRON_IN_DEV=1` дозволяє його в `NODE_ENV=development`. Ручний тригер: `GET /api/cron/{notifications,digest,backup}` (захищений `CRON_SECRET` якщо встановлений).